

+

TIPOS DE DATOS

Primitivos

string `"h" "m" `t``

number `42 3.14 NaN`

bigint `900n`

boolean `true / false`

null `ausencia intenc.`

undefined `sin asignar`

symbol `Symbol('id')`

Referencia

object `{} [] Date()`

function `fn() =>`

`typeof null // "object" ▲`

`typeof NaN // "number" ▲`

≡

VARIABLES

declaración

var **Global/func.**
hoisting — evitar

let **Bloque,**
reassignable

const **Bloque,**
NO reassignable

Asignación

`=` `asignar`
`+=` `suma y asigna`
`-=` `resta y asigna`
`*=` `/=` `%=` `mult / div / módulo`
`**=` `potencia`
`&&=` `||=` `??=` `asig. lógica`

Σ

ARITMÉTICOS

operadores

`+` `suma`

`-` `resta`

`*` `multiplicación`

`/` `división`

`%` `módulo (resto)`

`**` `potencia 2**3 8`

`++` `incremento a++ / ++a`

`--` `decremento`

Ejemplos rápidos

`5 == "5" true`
`5 === "5" false`
`NaN===NaN false`
`Object.is(NaN,NaN) true`

≈

COMPARACIÓN

`===` `igualdad estricta`
`!==` `desigualdad estricta`
`==` `igualdad loose`
`!=` `desigualdad loose`
`>` `mayor que`
`<` `menor que`
`>=` `mayor o igual`
`<=` `menor o igual`

Operadores lógicos

`&&` `AND — ambos true`
`||` `OR — uno true`
`!` `NOT — niega`
`??` `Nullish coalescing`
`&&=` `||=` `??=` `asig. lógica`

⚙️

OTROS OPERADORES

`? :` `ternario; cond ? a : b`
`?.` `optional chaining`
`??` `nullish coalescing`
`...` `spread / rest`
`typeof` `tipo string`
`instanceof` `comprueba prototipo`
`in` `key en objeto/array`
`delete` `borra propiedad`
`void` `evalúa undefined`
`await` `espera promesa`

Bitwise

`&` `AND`
`|` `OR`
`^` `XOR`
`~` `NOT`
`<<` `shift left`
`>>` `shift right`
`>>>` `unsigned`

⇒

CONVERSIÓN

de tipos

`Number(x)` `a número`

`String(x)` `a string`

`Boolean(x)` `a boolean`

`parseInt(str)` `entero`

`parseFloat(str)` `decimal`

`+x` `a número`

`!!x` `a boolean`

`Number("42")` `42`

`Boolean(0)` `false`

`Boolean("")` `false`

`Boolean("hi")` `true`

☀️

TRUTHY / FALSY

Falsy — se convierten en false:

`false`

`0`

`-0`

`0n`

`""`

`null`

`undefined`

`NaN`

Todo lo demás es Truthy

🕒

SCOPE

& hoisting

`Global` `window / globalThis`

`Módulo` `solo en el archivo .js`

`Función` `dentro de function()`

`Bloque` `{}` `con let / const`

Hoisting

`var` `sube declaración — valor no`
`function` `completa — usable antes`
`let/const` `TDZ — ReferenceError`

ERRORES COMUNES — Tipos, Variables y Operadores

⚠️

`// Closure pattern (1,2,3,4,5)`
`function (x2==0) {`
 `let count = 0;`
 `map(n => n+2)`
 `return`
 `== en lugar de ===`
 `count, val: () => count }; .reduce((a,b)=>a+b,0)`
 `0 == "" → true / null == undefined → true`
 `const NO es immutable`
 `const obj = {}; obj.x = 1; // está Sí funciona`
`}`

`const PI = Math.PI;`
`// 3.14159...`

`arr.sort((a,b) => a-b);`
`// ordena correctamente`

`val ?? 'default'`
`// nullish coalescing`

`Number.isNaN(NaN) // true`
`// NO usar isNaN()`

`// async/await`
`async function load(url){`
 `try {`
 `const r:= await fetch(url);`
 `return await r.json();`
 `} catch(e) { console.error(e); }`
`}`

`typeof null "object"`
`bug histórico de JS — null no es un objeto`

`NaN no es igual a nada`
`NaN === NaN → false`
`usar: Number.isNaN(x)`

JavaScript Quick Reference

¿Cuándo usar const? ¿Por qué === en lugar de ==? ¿Qué es TDZ?

Cap. 01 · Pág. 1

f STRINGS

métodos clave

3 formas de declarar:

```
"doble" // clásica
'simple' // igual que doble
`template` // permite ${expr}
```

Métodos — todos devuelven nuevo valor:

```
.length -> num cantidad de caracteres
.toUpperCase() -> str todo mayúsculas
.toLowerCase() -> str todo minúsculas
.trim() -> str quita espacios
.trimStart() -> str solo inicio
.trimEnd() -> str solo fin
.includes(str) -> bool ¿contiene?
.startsWith(str) -> bool ¿empieza con?
.endsWith(str) -> bool ¿termina con?
.indexOf(str) -> num posición o -1
.lastIndexOf(str) -> num última posición o -1
.at(n) -> char .at(-1) = último
.slice(i, j) -> str extrae — NO modifica
.split(sep) -> arr divide en array
.replace(a, b) -> str reemplaza 1ª coincidencia
.replaceAll(a, b) -> str reemplaza todas
.padStart(n, c) -> str rellena al inicio
.padEnd(n, c) -> str rellena al final
.repeat(n) -> str repite n veces
.match(regex) -> arr busca con regex
.matchAll(regex) -> iter todas las coincidencias
.concat(...) -> str concatena
```

Template Literal:

```
const n = "Oscar";
const msg = `Hola ${n}!`;
// → 'Hola Oscar!'
```

Σ NUMBER & MATH

métodos clave

```
Number.isNaN(x) — ¿es NaN?
Number.isFinite(x) — ¿es finito?
Number.isInteger(x) — ¿es entero?
Number.parseInt(str) — entero
Number.parseFloat(str) — decimal
x.toFixed(2) — 2 decimales fijos
x.toString(base) — base 2/8/16
x.toPrecision(n) — n dígitos
```

Math

```
Math.round(x) — redondea
Math.floor(x) — hacia abajo
Math.ceil(x) — hacia arriba
Math.trunc(x) — quita decimales
Math.abs(x) — valor absoluto
Math.max(a,b,c) — el mayor
Math.min(a,b,c) — el menor
Math.random() — 0 ≤ n < 1
Math.pow(x, n) — = x*n
Math.sqrt(x) — raíz cuadrada
```

Math.PI — 3.14159...

Math.E — 2.71828...

Valores especiales

```
NaN Not a Number — typeof 'number'
Infinity 1/0 Infinity
Number.EPSILON diferencia mínima
```

Entero aleatorio:

```
Math.floor(
  Math.random()*(max-min+1)
) + min;
```

☰ ARRAYS

métodos esenciales

Crear / acceder

```
const a = [1, 2, 3];
a[0] // 1 a.at(-1) // 3
a.length // 3
const [x,...r] = a;
```

Iterar

```
.forEach(fn) sin retorno
.map(fn) nuevo array
.filter(fn) filtrado
.reduce(fn,0) acumulado
.flatMap(fn) map+flat
Agregar/eliminar/modificar
.push(x) agrega al final
.pop() quita del final
.unshift(x) agrega al inicio
.shift() quita del inicio
.splice(i,n,x) elimina/inserta
.fill(v,i,j) rellena
```

Buscar

```
.find(fn) primer match
.findIndex(fn) índice o -1
.findLast(fn) último match
.indexOf(x) índice o -1
.includes(x) boolean
```

Verificar

```
.some(fn) ¿alguno?
.every(fn) ¿todos?
```

No modifican — copia

```
.slice(i,j) copia parcial
.concat(...) combinados
.flat(n) aplanado
.join(sep) string
.toSorted(fn) copia ordenada ES23
.toReversed() copia invertida ES23
.with(i,val) copia con cambio
```

Otros

```
.reverse() invierte
.sort(fn) ordena
Array.from(x) desde iterable
[...a,...b] spread/combina
```

// async/await

```
async function load(url){
  try {
    const r = await fetch(url);
    return await r.json();
  } catch(e) { console.error(e); }
}

.reverse() modifica el original
Usar .toReversed() para obtener copia sin mutar (ES2023)

indexOf(NaN) siempre -1
[NaN].indexOf(NaN) → -1 usar: .includes(NaN) → true
```

```
// Closure pattern (1,2,3,4,5)
function f(n){
  let count = 0;
  return {
    // [1,10,2].sort() → [1,10,2] 'correcto: .sort((a,b) => a-b)'
    // .splice() modifica / .slice() NO
    // splice(1,1) muta el array — slice(1,2) devuelve copia nueva
  };
}
```

```
[1,10,2].sort((a,b)=>a-b) // [1, 2, 10]
.slice() // NO modifica
.splice() // SÍ modifica
```

```
str.at(-1) // último char
Array.from('hola') // → ['h','o','l','a']
// ES2022+
```

CONDICIONALES

```
if (condición) {
  // verdadero
} else if (otra) {
  // alternativa
} else {
  // por defecto
}
```

Ternario:

```
cond ? 'si' : 'no'
```

Switch:

```
switch (valor) {
  case 'a':
    break;
  default: ...
}
```

BUCLES

for clásico

```
for (let i=0; i<n; i++) {
  // i: 0,1,...,n-1
}
```

for...of — valores

```
for (const item of array) {
  // valor
}
```

for...in — claves

```
for (const key in objeto) {
  // clave
}
```

while

```
while (condición) {
  // repite
}
```

do...while

```
do {
  // min 1 vez
} while (cond);
```

Control en bucles:

break sale del bucle

continue salta iteración

return sale de la función

FUNCIONES

Declaración — hoisted

```
function suma(a, b) {
  return a + b;
}
// usable ANTES de declararse
```

Expresión — no hoisted

```
const suma = function(a, b) {
  return a + b;
};
```

Arrow function — más común

```
const fn = (a, b) => {
  return a + b;
};
// retorno implícito:
const fn = (a, b) => a + b;
// 1 param sin ():
const doble = x => x * 2;
```

Parámetros especiales

// Por defecto:

```
function f(x = 10) {}
// Rest:
function f(...args) {}
// Destructuring:
function f({ a, b }) {}
```

Callback

```
function run(fn) { fn(); }
run(() => console.log('!'));
```

Funciones de orden superior

// recibe o devuelve funciones

```
const apply = (fn, x) => fn(x);
```

Closure

```
function contador() {
  let n = 0;
  return () => ++n;
}
const c = contador();
c(); // 1 c(); // 2
```

IIFE

```
(function() { ... })();
(() => { ... })();
```

OBJETOS

Crear / acceder

```
const user = {
  name: 'Oscar',
  age: 25,
  greet() { return 'Hi!'; },
};
user.name // dot
user["age"] // bracket
```

Shorthand & Computed

```
const obj = { name }; // shorthand
const o = { [key]: 25 }; // computed
```

Object — métodos estáticos

```
Object.keys(obj) // → claves[]
Object.values(obj) // → valores[]
Object.entries(obj) // → [[k,v],...]
Object.assign({}, a) // copia shallow
{...obj} // spread copia
Object.freeze(obj) // immutable
Object.hasOwn(o,k) // ¿tiene key?
structuredClone(obj) // deep clone
```

Optional chaining & Nullish

```
user?.address?.city // sin error
user?.fn?.() // llama si existe
val ?? 'default' // null/undef
```

DESTRUCTURING & SPREAD

Array

```
const [a, b] = [1, 2];
const [x, ...rest] = arr;
const [, tercero] = arr;
```

Object

```
const { name, age } = user;
const { name: alias } = user;
const { x = 10 } = obj;
const { a, ...resto } = obj;
```

Spread

```
const copy = [...arr, 4, 5];
const obj2 = { ...obj, x: 1 };
const merged = { ...a, ...b };
```

Rest

```
function f(a, ...rest) {
  // rest es array
}
```

Spread en arrays

```
const arr = [1, 2, 3, 4, 5];
const copy = [...arr, 6, 7, 8];
```

Spread en objetos

```
const obj1 = { a: 1, b: 2 };
const obj2 = { c: 3, d: 4 };
const merged = { ...obj1, ...obj2 };
```

Spread en funciones

```
function f(a, b, c) {
  // a, b, c son arrays
}
```

Spread en bucles

```
for (const [key, value] of Object.entries(obj)) {
  // key, value son arrays
}
```

Spread en condicionales

```
const { a, b } = { a: 1, b: 2 };
const { a, b } = { a: 1, b: 2 };
const { a, b } = { a: 1, b: 2 };
```

Spread en async/await

```
async function load(url) {
  try {
    const r = await fetch(url);
    return await r.json();
  } catch(e) { console.error(e); }
}
```

Spread en for...of

```
for (const [key, value] of Object.entries(obj)) {
  // key, value son arrays
}
```

Spread en for...in

```
for (const key in obj) {
  // key es string
}
```

// Closure pattern (1,2,3,4,5)

```
function f() {
  let count = 0;
  return () => count++;
}
const c = f();
c(); // 1
c(); // 2
```

ERRORES COMUNES — Funciones, Bucles y Objetos

Arrow fn no tiene 'this' propio

```
const o={f:()=>this} → this = scope externo, NO el objeto
```

Olvidar return en arrow con bloque

```
const f = x => { x * 2 } → undefined (falta return)
```

```
const doble = x => x * 2;
// arrow implícita
```

```
((() => { ... })());
// IIFE
```

```
obj?.prop?.sub ?? 'def'
// chaining + nullish
```

```
for...of // valores
for...in // claves
```



CLASES

ES6+

```
class Animal {
  #name; // privado ES2022
  static count = 0;

  constructor(name) {
    this.#name = name;
    Animal.count++;
  }

  get nombre() { return this.#name; }
  set nombre(v) { this.#name = v; }

  hablar() {
    return `Hola, soy ${this.#name}`;
  }

  static crear(n) { return new Animal(n); }
}

class Perro extends Animal {
  constructor(n) { super(n); }
  hablar() {
    return super.hablar()+' Guau!';
  }
}

const p = new Perro('Rex');
p.hablar() // 'Hola, soy Rex Guau!'
p instanceof Animal // true
```

MÓDULOS (ES Modules)

```
export const PI = 3.14;
export default function main() {}
export { suma, PI };
import main from './math.js';
import { suma } from './math.js';
import * as M from './math.js';
```

CONSOLE & UTILS

```
console.log(x) — imprimir
console.warn(x) — advertencia
console.error(x) — error
console.table(arr) — tabla visual
JSON.stringify(x) — string JSON
JSON.parse(str) — objeto JS
```



PROMISES

& async/await

```
Promise — base de lo async
const p = new Promise(
  (resolve, reject) => {
    resolve('ok'); // éxito
    reject('err'); // fallo
  }
);

.then / .catch / .finally
p.then(val => console.log(val))
  .catch(err => console.error(err))
  .finally(() => cleanup());

Promise estáticos
Promise.all([p1,p2]) // todos o fallo
Promise.allSettled([...]) // nunca falla
Promise.race([p1,p2]) // el más rápido
Promise.any([p1,p2]) // 1º exitoso

async / await
async function getData() {
  try {
    const res = await fetch('/api');
    const data = await res.json();
    return data;
  } catch (err) {
    console.error(err);
  } finally {
    // siempre
  }
}

Manejo de errores
try { /* riesgoso */ }
catch (err) { err.message }
finally { /* siempre */ }
throw new Error('msg');
```



DOM

manipulación

```
Seleccionar
document.getElementById('id')
document.querySelector('.cls')
document.querySelectorAll('p')
el.closest('.parent')

Modificar
el.textContent = 'texto';
el.innerHTML = '<b>x</b>';
el.style.color = 'red';
el.classList.add('cls');
el.classList.toggle('cls');
el.setAttribute('href','/');
el.dataset.userId // data-*

Crear / eliminar
document.createElement('div');
parent.appendChild(el);
el.insertAdjacentHTML('afterend','<p>');
el.remove();

Eventos
el.addEventListener(
  'click', (event) => {
    event.preventDefault();
    event.target;
  }
);
// click dblclick mouseenter
// keydown keyup input change
// submit focus blur load
```

ES2024+ — Características clave

```
Optional chaining: obj?.prop?.sub
Nullish coalescing: val ?? 'default'
Logical assignment: x ??= 'v'   x ||= 'v'
structuredClone: deep clone nativo
crypto.randomUUID(): // uuid v4
Object.groupBy(arr,fn): // agrupar ES2024
Array.fromAsync(): // async iter.
AbortController: // cancelar fetch
```

★ TIP PROFESIONAL

Usa '===' siempre. Nombres descriptivos.
Funciones pequeñas con un solo propósito.

// Array chain

```
// Closure pattern (1,2,3,4,5)
function ERRORES COMUNES — Clases, Async y DOM (n => n*2==10)
  let count = 0; // cap(n => n*2)
  return { // await fuera de async
    count, val: () => count }, .reduce((a,b)>>a+b,0)
  }
  // SyntaxError — await SÓLO funciona dentro de async function
  Promise sin return en .then()
  .then(x => { x*2 }) → undefined (falta return o usar =>)
```

```
p instanceof Animal // true
// herencia verificada
```

```
await fetch('/api')
// solo en async fn
```

```
Promise.allSettled([])
// nunca falla
```

```
el.classList.toggle('active')
// añade o quita
```